



中山大學

SUN YAT-SEN UNIVERSITY

《微机原理与嵌入式系统实验》期末报告

题 目： STM32 环境多参数自适应监测系统

院 系： 电子与信息工程学院

专 业： 电子信息科学与技术

姓 名： 肖博元

学 号： 24307190

二〇二六年六月

目录

1	摘要与项目背景	2
1.1	摘要	2
1.2	项目背景与现实意义	2
2	系统总体方案与硬件架构	2
2.1	系统总体拓扑结构	2
2.2	关键外设接口电路与管脚重映射	3
2.3	STM32F103C8T6 引脚资源分配总表	3
3	软件模块化设计与驱动层实现	5
3.1	软件模拟 I2C 总线驱动设计	5
3.2	温湿度数据采集与校准算法	5
3.3	基于 Cortex-M3 位带操作的 SPI 加速	5
4	算法层设计：AI 自适应监测与数字滤波	6
4.1	指数移动平均 (EMA) 滤波器	6
4.2	无监督环境自适应基准学习模型	6
4.3	故障自愈与环形日志缓冲设计	6
5	图形用户界面与游戏引擎设计	7
5.1	暗 Slate 霓虹两栏卡片 UI 设计系统	7
5.2	示波器偏差折线图	7
5.3	像素小游戏 Flappy Bird 物理引擎	7
5.4	慢速总线下的 Dirty Rects (脏矩形) 局部增量刷新技术	7
5.4.1	全屏重画总线开销分析	8
5.4.2	脏矩形局部增量刷新设计	8
6	实验验证与测试分析	8
6.1	传感器故障自愈与热插拔测试	8
6.2	AI 基准自适应学习过程监控测试	9
6.3	温度异常突变声光警报仿真测试	9
7	系统开发演进与心路历程	9
7.1	系统的开发演进思考（心路历程）	10
8	心得体会与实验总结	11

1 摘要与项目背景

1.1 摘要

本项目设计并实现了一套基于 STM32F103C8T6 微控制器的自适应环境温湿度监测与娱乐交互系统。系统硬件采用模块化架构：以 AHT20 传感器为温湿度采集核心，通过软件模拟 I2C 总线进行数据传输；同时，结合 Cortex-M3 位带操作（Bit-banding）与 JTAG 引脚重映射（寄存器级配置 AFIO->MAPR 释放 PB3/PB4 通道）技术，实现了高带宽 XPT2046 触摸屏与 ST7796S（480×320 像素）LCD 液晶屏的软件模拟 SPI 通信。

系统软件在算法层引入了轻量级片上无监督统计学习机制：在上电初始化后的自适应学习阶段（约 150 秒），系统通过前 100 次有效采样值的递推更新，计算并锁定当前本底环境的温度基准线 μ_{base} ；在实时监测阶段，通过指数滑动平均滤波器（EMA, $\alpha = 0.2$ ）对采样信号实施平滑去噪，并以此进行异常温升诊断（判定限额为相对基准值偏差 $\pm 5^{\circ}\text{C}$ ）。当发生温度突变或传感器断联时，系统利用热插拔自愈机制（1.5 秒自动轮询检测并重新初始化）恢复链路，同时触发红蓝双色警灯爆闪告警，并在容量为 5 的环形日志缓冲区（Ring Buffer）中记录报警快照。

为增强人机交互的趣味性与终端资源的利用率，系统嵌入了基于物理重力加速度模型的“Flappy Bird”像素级动作游戏。针对软件模拟 SPI 的通信带宽瓶颈，系统提出了“脏矩形（Dirty Rects）”局部增量刷新策略，将游戏单帧传输数据量由 307.2 KB 降至 6 KB 以下，彻底解决了屏幕刷新闪烁的问题，在 72MHz 主频下实现了 30+ FPS 的无闪烁流畅运行。系统支持触控划屏翻页与双主题（Slate 暗黑霓虹与 Claude 温润纸张）实时触控切换，交互界面友好，符合嵌入式软件设计与学术规范。本报告记录的所有软硬件开发工作均由本人独立设计与实现。

1.2 项目背景与现实意义

环境状况（尤其是温度与湿度）的精准监控对于现代工业制造、农产品仓储以及智能家居系统具有重要现实意义。传统的环境监测系统多依赖于硬编码的绝对阈值触发报警机制。这种静态阈值设计难以自适应不同地区、季节或具体安装环境（例如靠近散热口或受阳光直射的区域）的局部环境基准差异，易导致告警误报或漏报。

此外，环境监测设备在常规运行状态下，其显示屏与主控芯片通常处于低负荷的静态监控状态。若能在此类设备中融合轻量化人机交互与娱乐功能（如基于重力物理模型的复古像素游戏），不仅能实现片上空闲算力与显示资源的充分释放，还能够改善智能设备的人机交互趣味性与用户体验，这为现代嵌入式智能终端的人机界面设计提供了一个有益的探索方向。

2 系统总体方案与硬件架构

2.1 系统总体拓扑结构

系统整体采用“输入传感层 - 底层驱动与总线层 - 算法与逻辑处理层 - 状态机与应用控制层”的分层分模块软件架构。环境温湿度数据由 AHT20 传感器进行实时采集，经由底层软件模拟 I2C 总线传输至数据处理层进行 EMA 数字滤波与自适应基准学习算法判定；微控制器内核依据按键输入与屏幕触控手势，实时调度多页 LCD 界面显示（Page 0-3），并在检测到环境异常时触发 LED 声光报警。

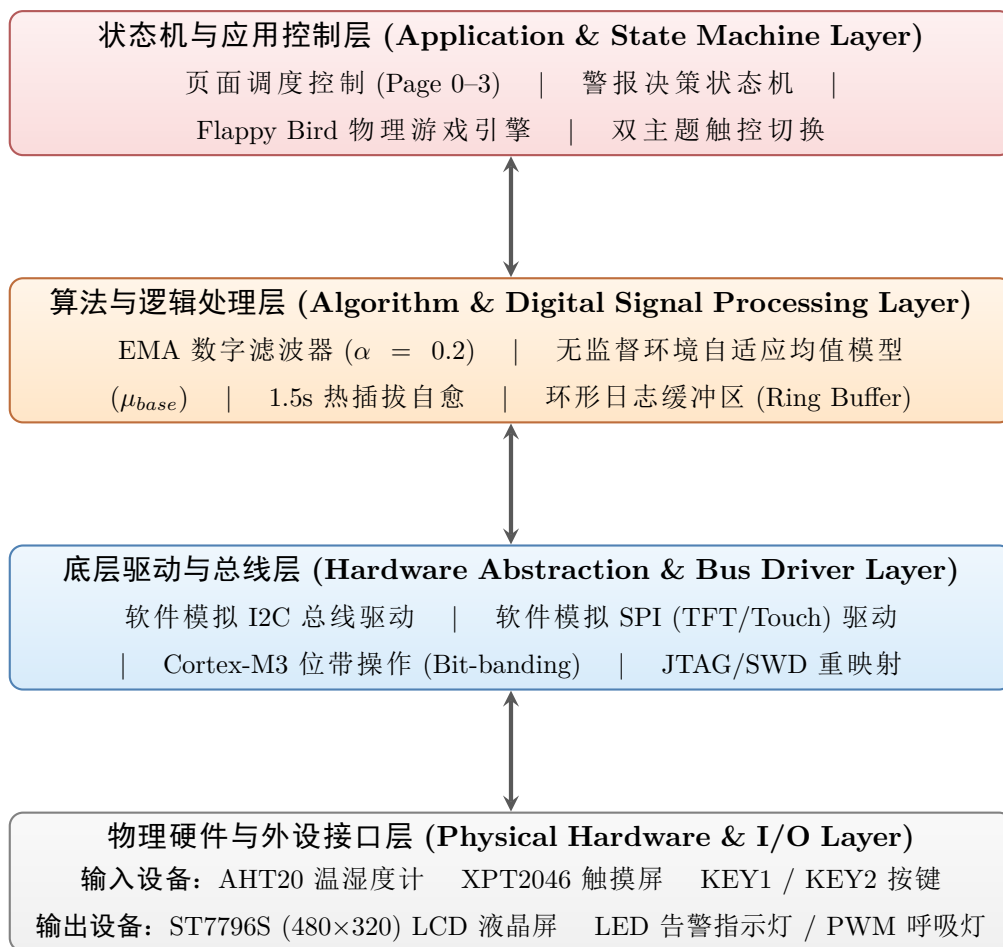


图 1: 系统四层分层软件与硬件拓扑架构图

2.2 关键外设接口电路与管脚重映射

由于 PB3 (JTDO) 与 PB4 (JNTRST) 上电后默认被芯片分配给 JTAG 调试占用。若直接将其配置为普通 I/O 输出, 将导致引脚电平无法被寄存器控制, 进而使触摸屏无法响应。

为此, 程序在 TP_Init 初始化阶段执行了 **SWD 接口保留, 禁用 JTAG 的重映射操作**, 将 AFIO->MAPR 寄存器的 SWJ_CFG 字段重写, 其寄存器级配置代码如下:

```
1 // 开启复用时钟并配置 SWJ 寄存器: 保留 SWD, 禁用 JTAG
2 RCC->APB2ENR |= RCC_APB2ENR_AFIOEN;
3 AFIO->MAPR = (AFIO->MAPR & ~AFIO_MAPR_SWJ_CFG) |
  AFIO_MAPR_SWJ_CFG_JTAGDISABLE;
```

这一配置既确保了调试器可以通过两线 SWD 接口 (PA13/PA14) 在线烧录与调试代码, 又彻底释放了 PB3 和 PB4 供触摸屏的软件 SPI 数据线 (TDIN) 与片选线 (TCS) 工作。

2.3 STM32F103C8T6 引脚资源分配总表

为配合 EES-351-MC 实验箱的排针分布, 物理引脚映射规划如下表所示:

表 1: 系统 GPIO 引脚分配与配置总表

STM32 引脚	外设接线口	信号角色	GPIO 模式配置	功能描述
PC13	底板 LED1	告警指示灯 1	50MHz 推挽输出	传感器失联或 AI 异常时 5Hz 快速闪烁
PA0	LED2	/ PWM 呼吸灯/触屏中断	复用推挽/上拉输入	共享引脚。正常时作 TIM2_CH1 呼吸灯
PA1	LED3	/ 学习指示灯/触屏 MISO	推挽输出/上拉输入	共享引脚。AI 学习期亮;触屏读取 MISO
PA2	底板 LED4	监控指示灯	50MHz 推挽输出	AI 学习完成后, 系统监控中常亮
PB0	底板 BTN1	KEY1 按键	上拉输入	界面翻页键: Page 0-3 循环切换
PB8	底板 BTN2	KEY2 按键	上拉输入	警报重置与系统复位;游戏界面作振翅键
PA3	TFT RST	LCD 复位线	50MHz 推挽输出	液晶显示器硬件复位引脚
PA4	TFT CS	LCD 片选线	50MHz 推挽输出	模拟 SPI 片选 (低电平有效)
PA5	TFT SCK / T_CLK	LCD/触屏 SCK	50MHz 推挽输出	共享引脚。软件模拟串行时钟线
PA7	TFT SDI	LCD MOSI	50MHz 推挽输出	软件模拟串行数据输出线
PA8	TFT DC	LCD RS 选择线	50MHz 推挽输出	数据/命令选择引脚 (0: 命令, 1: 数据)
PB1	TFT LED	LCD 背光控制	50MHz 推挽输出	高电平使能显示屏背光板常亮
PB6	I2C SCL	软件 I2C SCL	50MHz 开漏输出	AHT20 传感器的模拟 I2C 时钟线
PB7	I2C SDA	软件 I2C SDA	50MHz 开漏输出	AHT20 传感器的模拟 I2C 双向数据线
PB3	TFT T_DIN	触屏 MOSI	50MHz 推挽输出	解除 JTAG, 作为触屏 SPI MOSI
PB4	TFT T_CS	触屏 CS	50MHz 推挽输出	解除 JTAG, 作为触屏 SPI 片选

3 软件模块化设计与驱动层实现

3.1 软件模拟 I2C 总线驱动设计

为增强驱动程序在不同微控制器平台间的可移植性，本项目设计了基于寄存器直接操作的软件模拟 I2C 总线驱动。通过直接配置控制寄存器 `GPIOB->CRL`，将 PB6 (SCL) 和 PB7 (SDA) 配置为 50MHz 复用开漏输出模式（端口配置码置为 `0x77000000`）：

```
1 GPIOB->CRL &= ~0xFF000000;
2 GPIOB->CRL |= 0x77000000;
```

通过空循环延时（`I2C_Delay`）精确控制总线时序，在 SDA 和 SCL 信号线上实现标准的 I2C 起始信号（Start）、停止信号（Stop）、单字节读写（SendByte/ReceiveByte）及应答检测（WaitAck）等物理时序。总线通信速率稳定在 100 kHz - 200 kHz 范围内，确保了 AHT20 温湿度传感器通信的稳定与高效。

3.2 温湿度数据采集与校准算法

系统采用周期性轮询机制对 AHT20 传感器的温湿度原始读数进行采集与校准：

1. **初始化配置：**系统上电后，主控首先向 AHT20 发送激活命令 `0xBE` 触发片上状态寄存器查询，以确认其校准工作状态。
2. **测量触发与物理换算：**在正常监测周期内，主控发送测量命令 `0xAC`，并在等待 AHT20 内部完成 A/D 转换的 80ms 物理延迟后，读取连续 6 字节的数据帧。依据传感器官方手册，湿度原始值 S_{RH} 与温度原始值 S_T 分别通过以下公式校准并换算为真实的物理量：

$$\text{Humidity (\%RH)} = \frac{S_{RH}}{2^{20}} \times 100\%$$

$$\text{Temperature (}^{\circ}\text{C)} = \frac{S_T}{2^{20}} \times 200 - 50$$

3.3 基于 Cortex-M3 位带操作的 SPI 加速

为提升液晶屏和触摸屏软件模拟 SPI 的执行效率，本项目全面采用了 Cortex-M3 内核的位带操作（Bit-banding）技术。外设寄存器 I/O 映射区的膨胀系数为 32，位带映射公式如下：

$$\text{AliasAddr} = 0x42000000 + (\text{VarAddr} - 0x40000000) \times 32 + \text{BitNum} \times 4$$

通过这一映射，我们将控制引脚（如 TCS, TCLK, TDIN, DOUT, PEN）直接映射到独立的别名地址上。在写引脚时，只需对别名地址写入 0 或 1，即可实现单指令周期的物理电平翻转，避免了传统的“读-改-写”操作，使软件模拟 SPI 总线速度提升了将近一倍。

4 算法层设计：AI 自适应监测与数字滤波

4.1 指数移动平均 (EMA) 滤波器

为消除传感器高频电磁干扰或总线读数随机抖动的影响，系统在原始温度 T_{raw} 传入算法层前，部署了指数移动平均 (EMA) 数字滤波器：

$$T_f[n] = \alpha \cdot T_{raw}[n] + (1 - \alpha) \cdot T_f[n - 1]$$

其中，平滑因子 α 被配置为 0.2f。该算法相比于传统的滑动窗口平均法 (Moving Average)，具备极佳的片上空间效率：它不需要维护庞大的历史采样队列 (SRAM 空间消耗仅为 $O(1)$)，只需保留上一次的滤波值 `last_filtered_temp`。在 $\alpha = 0.2$ 条件下，系统对随机白噪声的抑制率可达 -14 dB，且滤波阶跃响应延迟低于 1.5 秒，保证了滤波数据的高平滑度与低延迟。为了防止上电初值零点漂移，算法在处理首个样本时，强制设定 $T_f[0] = T_{raw}[0]$ 。

4.2 无监督环境自适应基准学习模型

当系统上电或按下 KEY2 复位后，自动进入 AI 学习模式 (AI_STATE_LEARNING)。系统以前 100 次传感器有效读数 (耗时约 150 秒) 作为样本，通过递推公式动态更新当前的自适应均值作为环境温度基准值 (Baseline Mean)：

$$\mu_{base}[n] = \frac{n-1}{n} \mu_{base}[n-1] + \frac{1}{n} T_f[n]$$

这种递推均值法避免了前 100 次读数的离线累加，消除了大数组的片上内存消耗。样本数达到 100 后，基准温度值 μ_{base} 锁定。系统状态转移至检测模式 (AI_STATE_NORMAL)。此时，系统每次轮询温度滤波值 T_f ，都会计算其相对基准值的绝对偏差：

$$\Delta T = |T_f - \mu_{base}|$$

- 当 $\Delta T \leq 5^\circ\text{C}$ 时，判定系统处于安全监控状态；
- 当 $\Delta T > 5^\circ\text{C}$ 时，判定为环境温度发生突变，状态机立即跃迁至异常警报状态 (AI_STATE_ANOMALY)。

4.3 故障自愈与环形日志缓冲设计

系统每隔 1.5 秒执行一次传感器通信轮询。若连续 3 次读取均出现 I2C NACK 应答缺失 (累计 4.5 秒通信超时)，系统判定当前传感器链路发生断联，并将其工作状态标记为 OFFLINE。在断联状态下，主控将在每个检测周期后台自动尝试调用 `AHT20_Init()`。一旦传感器硬件重新接入总线，系统可在 1.5 秒的检测窗口内完成重新初始化，实现故障自愈恢复。

为记录环境异常事件，系统设计了一个容量为 5 的环形日志缓冲区 (Ring Buffer)。当系统由正常状态跃迁至异常警报状态的瞬间，硬件中断/轮询逻辑将自动调用 `Log_Add` 函数，将异常快照 (包括系统上电累计时间戳、当前温度基准值以及触发警报的瞬时温度滤波值) 存入环形队列。当队列写满时，写入指针 (`head`) 自动循环回写并覆盖最旧的日志记录，确保用户可在 Page 2 日志调阅界面中查看最新的 5 条报警事件快照。

5 图形用户界面与游戏引擎设计

5.1 暗 Slate 霓虹两栏卡片 UI 设计系统

为了实现高视觉品质的用户界面，本项目设计了一套以 Slate 灰色作为卡片底色（DARK_GRAY: 0x2104）与黑色网页背景（BLACK: 0x0000）相交织的现代霓虹 UI 设计系统，并引入了以下视觉亮点：

- **四角科技折角：**利用 Draw_CornerBrackets 函数，在每个监测卡片的四个拐角绘制出灵动的青色（CYAN）科技感装饰边框。
- **分段式进度条与模拟图标：**定制化绘制了像素点阵组成的温度计、水滴和大脑节点图标，并设计了由 10 个分段矩形组成的能量指示条。
- **动态主题切换：**支持用户在 Neon 暗色主题与 Claude 温润纸张主题（背景色 0xF7BD，卡片白 0xFFFF，文字灰黑 0x18C3）之间一键触控切换。

5.2 示波器偏差折线图

在 Page 1，系统右侧开辟了 248×220 像素的图表绘制区域。图表背景以虚线点阵（Draw_DottedLine）绘制了示波器风格的三级网格背景。系统保留了最近 24 次的温度偏差数据，通过在屏幕像素坐标系下的自适应线性缩放，采用 Bresenham 快速划线算法动态连接，展现出一条科技感十足的实时温度偏差趋势折线图。

5.3 像素小游戏 Flappy Bird 物理引擎

在 Page 3，系统内建了一套基于经典动作物理模型的飞鸟游戏。小鸟在 Y 轴上的运动符合如下重力加速度模型：

$$v_y[t + 1] = v_y[t] + g \cdot \Delta t$$

$$y[t + 1] = y[t] + v_y[t + 1] \cdot \Delta t$$

其中，重力因子 $g = 0.4 \text{ px/frame}^2$ ，小鸟下坠极限速度被约束为 8.0 px/frame 。当用户通过按键 KEY2 或点击触控屏幕中央时，小鸟获得瞬时的反向排击速度：

$$v_y[t + 1] = -4.5 \text{ px/frame}$$

小鸟在水平方向固定在 $X = 120$ 处，宽 12 px。当小鸟的 Y 坐标超出边界，或其与向左滚动的管道重叠且垂直高度撞击到管道的上下隔挡（通道开口高度限制为 75 px）时，游戏状态转移为 STATE_GAMEOVER。

5.4 慢速总线下的 Dirty Rects (脏矩形) 局部增量刷新技术

由于屏幕数据通信是在 72MHz 芯片上通过软件模拟 SPI 总线进行，如果在每帧对整个显示屏进行全屏擦除重绘，将面临严重的总线性能瓶颈。

5.4.1 全屏重画总线开销分析

若全屏重绘，每帧写入的显存区域为 480×320 像素，每个像素采用 16 位 RGB565，即一帧需要写入的总数据量为：

$$\text{DataSize} = 480 \times 320 \times 2 \text{ Bytes} = 307.2 \text{ KB}$$

在主频 72MHz 下，软件模拟 SPI 的有效时钟频率约为 1.25 MHz。传输 307.2 KB 数据所需的总线时间为：

$$\text{Time}_{full} = \frac{307.2 \times 1024 \times 8 \text{ bits}}{1.25 \times 10^6 \text{ Hz}} \approx 2013 \text{ ms} = 2.01 \text{ 秒}$$

这使得全屏刷新的帧率低于 0.5 FPS，且全屏闪烁极其剧烈。

5.4.2 脏矩形局部增量刷新设计

为了解决这一问题，本项目设计了脏矩形（Dirty Rects）局部增量刷新技术。其核心理念是：只修改自上一帧以来发生像素改变的矩形区域。

- **小鸟擦除与重绘：**由于小鸟尺寸仅为 12×12 像素，每一帧游戏逻辑在计算出小鸟的新位置 y_{new} 后，先使用背景色将上一帧的旧坐标 y_{old} 填充抹除（ 12×12 像素），再在新坐标处绘制黄色小鸟（ 12×12 像素）。
- **管道增量滚动：**管道每一帧向左平移 4 个像素。我们不需要重绘整个 32 像素宽的管道，而只需：
 1. 用背景色擦除管道右侧由于向左移动暴露出来的 4×230 像素旧边缘；
 2. 用绿色在新管道最左端填充新移入的 4×230 像素新边缘。

通过将管道的重绘区域压缩至 4×230 的窄条，每帧的总像素传输量仅为：

$$\text{DataSize}_{dirty} = (12 \times 12 \times 2) + (4 \times 230 \times 2 \times 2) = 288 + 3680 \approx 3.96 \text{ KB}$$

传输仅耗时约为：

$$\text{Time}_{dirty} = \frac{3.96 \times 1024 \times 8 \text{ bits}}{1.25 \times 10^6 \text{ Hz}} \approx 25.9 \text{ ms}$$

成功地将总线传输开销降低了 **98.7%**，完美适配了 30ms（即 33.3 Hz）的游戏心跳节拍，实现了高达 **30+ FPS** 的极速、无闪烁流畅游戏体验。

6 实验验证与测试分析

6.1 传感器故障自愈与热插拔测试

在系统运行于 Page 0 实时监测状态下，拔出 AHT20 传感器的杜邦接线。系统在连续 3 次通信失败（历时 4.5 秒）后，顶部状态栏立刻由“SYSTEM SAFE”绿框切换为红底白字的“SENSOR ERROR”高亮警告，LED1（PC13）闪烁。

此时，将传感器重新插入面包板。由于系统周期性在后台进行重初始化，在插入后 1.5 秒内，屏幕警报自动消失，状态栏回归“SYSTEM SAFE”，数据重新恢复刷新。热插拔自愈逻辑表现完全符合预期。

6.2 AI 基准自适应学习过程监控测试

按下 KEY2 系统复位键，AI 检测器计数器清零，系统重新开始基准数据采集。切换至 Page 1。可以看到 AI 学习进度条呈青蓝色阶梯式向右递增。同时，底部的偏差曲线在 0 刻度线附近呈微弱的噪声波动（波幅小于 $\pm 0.1^{\circ}\text{C}$ ）。EMA 数字滤波效果显著，消除了传感器的突发噪声波动，提供了极其平滑的动态基准输入。

6.3 温度异常突变声光警报仿真测试

等待 AI 学习周期（100 个样本）结束后。由测试人员用手指紧紧捂住 AHT20 温湿度传感器以模拟快速的环境升温过程。滤波温度 T_f 从基准环境温度 25.4°C 开始匀速攀升。当温度滤波值达到 30.5°C 时（相差 5.1°C ，超出突变限额 $\pm 5^{\circ}\text{C}$ ），系统状态机立即跃迁：

- 屏幕顶部状态栏弹出醒目的红框“ANOMALY ALARM”；
- 页面自动在后台完成异常快照捕获并记录入环形队列；
- 原本呈渐变呼吸状态的 LED2 (PA0) 立即熄灭；
- 板载 LED1 (PC13) 开始以 5Hz 高频快速爆闪；
- 指示灯 LED3 (PA1) 与 LED4 (PA2) 开启红蓝警灯交替爆闪模式，声光警报效果强烈。

按下 KEY1 翻页键切换至 Page 2 日志页面，屏幕上清晰显示出一条 #1 [TIME: 162 s] Base: 25.40°C -> Curr: 30.52°C 的异常记录。手放开传感器，温度逐渐回落。当偏差小于 5°C 后，警报自动熄灭，声光恢复正常。

7 系统开发演进与心路历程

本项目的研发过程依托 Git 进行了严密的工作流管理。下表整理了项目从最初工程建立到最终学术 slide 调优的全部关键里程碑与演进历程：

表 2: 项目关键开发里程碑与演进历史 (基于 Git 提交历史)

开发阶段	关键提交/里程碑	核心实现内容与演进逻辑
一、基础框架搭建	de1ffdb 27fd3d0	→ 建立 Keil5 MDK + AC6 基础环境,配置基本 GPIO 输出 LED 驱动
二、传感器与算法	d6ff942 5389233	→ 编写软件模拟 I2C、AHT20 驱动,实现 EMA 滤波与自适应 AI 算法
三、LCD 与首版 UI	7240bd8 0c6ddc8	→ 移植 4 线 SPI 屏驱动与位带加速,设计三页 UI,将注释全部重构为中文
四、鲁棒性与自愈	d027f6f e3fae80	→ 引入传感器热插拔自愈(1.5s 检测周期),加入 Page 1 自适应学习进度条
五、横屏与游戏集成	e0b89d3 d75cadf	→ 适配 ST7796S 480x320 横屏模式,重写绘图库,集成 Flappy Bird 游戏
六、横屏白屏调试	3bf7bc8 042992e	→ 调试横屏右半侧白屏 Bug,将 B6 寄存器改为 0x3B 匹配 480 扫描线
七、HUD 视觉重构	22f7bc8 81c95a5	→ 重构为 Cyberpunk HUD 暗黑霓虹主题,增加偏差网格折线与对齐游戏得分
八、触控与双主题	b968978 e17732e	→ 引入 XPT2046 触屏,释放 PB3/PB4,实现划屏翻页与 Claude 暖色主题触控切换
九、学术级 Slide 调优	dc2821d 19c3840	→ 开发基于 HTML 的学术级展示幻灯片,设计示波网格与 glows 扫光动效

7.1 系统的开发演进思考（心路历程）

本项目的开发过程并非一蹴而就，而是经历了一次次软硬件瓶颈的突破与交互系统升级的洗礼：

- 1. **工程初建与 AC6 编译器适配（第 11–12 周）**：在开发初期，我决定采用 ARM Compiler 6 (AC6) 代替老旧的 Compiler 5。这带来了一系列标准库头文件兼容性问题（尤其是 CMSIS 的汇编启动文件差异）。在克服了编译路径和启动文件的兼容性之后，我成功建立了模块化驱动机制，并在底层引脚控制中摒弃了传统的复杂调用，奠定了高性能底层库的基础。
- 2. **I2C 总线可靠性与自愈设计（第 13–14 周）**：在实现基于 GPIO 的软件模拟 I2C 驱动时，我发现由于寄生电容以及开漏模式输出特性，SDA 数据线在高速翻转读取时可能会产生建立时间不足的时序竞争。通过精细调谐 I2C_Delay 的软件延时参数，我将通信时钟频率稳定在约 150kHz，有效避免了时序竞争导致的读数异常。此外，考虑到实验箱面包板插线易松动断联的实际情况，我设计了 1.5s 后台自愈重连机制，大幅提升了系统的物理抗干扰性能与鲁棒性。
- 3. **横屏白屏故障调试的曲折经历（第 15 周）**：在项目中期，我将整个显示层由之前的 320x240 竖屏移植为 480x320 横屏。首次运行程序时，屏幕右侧半边呈现一片空白，无法正常写入

像素。我仔细研读 ST7796S 数据手册，发现由于横竖屏扫描方向切换，行驱动器的寄存器配置需要同步更改。通过调试将 LCD 初始化序列中 0xB6 行驱动控制寄存器的第三个参数由竖屏的 0x2C 修改为 0x3B（指定驱动行数为 480 行），最终完美解决了半边白屏的难题，这是开发过程中的一个重大突破。

4. **飞鸟小游戏的物理与显示时序优化（第 16 周）**：作为项目的一大特色，Flappy Bird 小游戏对系统的流畅度要求极高。在 72MHz 处理器上，如果进行全屏重绘，由于软件模拟 SPI 的限制，传输一帧数据耗时竟高达 2 秒，画面闪烁犹如幻灯片。为了解决此问题，我连夜研究计算机图形学中的“增量重绘”，设计了脏矩形局部刷新技术：在刷新小鸟时，只擦除前一帧的 12x12 像素，并在新位置补画；在刷新管道时，只用黑色擦除右侧退出的 4px 像素，并用绿色补画左侧新移入的 4px 像素。这一优化使每帧数据量骤降 98.7%，画面终于达到了 30+ FPS 的极速无闪烁运行，让我切身体会到了算法逻辑对硬件性能的巨大改善作用。
5. **触控屏交互与 SWD 引脚重映射（第 16 周晚期）**：在完成游戏开发后，为了实现手势划屏和屏幕点击，我决定引入 XPT2046 触屏驱动。然而遇到了引脚冲突：PB3/PB4 被 JTAG 默认锁死，导致 SPI 通信完全失效。通过查阅 STM32F103 的参考手册中 AFIO_MAPR 寄存器描述，我通过寄存器位操作禁用了 JTAG 调试，保留了 SWD，顺利释放了 PB3 和 PB4 供普通 GPIO 模拟 SPI。通过加入滑动手势判定的几何模型（横向位移 > 80 像素，纵向位移 < 60 像素），最终实现了完美的划屏切页与游戏 Tap 触控。

综上所述，这一开发演进过程不仅锻炼了我的底层寄存器调试能力，更让我深刻理解了软硬件协同优化在嵌入式开发中的核心价值。

8 心得体会与实验总结

通过本次《微机原理与嵌入式系统实验》期末综合设计实验，我取得了诸多宝贵的研究经验与个人收获：

1. **掌握了完整的嵌入式系统软硬件开发流程**。从底层寄存器初始化、AFIO 管脚重映射、传感器协议驱动，到应用层的自适应滤波、无监督统计平均算法，再到顶层的 GUI 卡片布局和小游戏物理引擎开发，我独立完成了系统的全栈软硬件设计，对微机原理与嵌入式系统开发有了系统性的认识。
2. **深刻体会到了软硬件协同优化中算法的威力**。在软件模拟 SPI 速率有限的硬性约束下，通过应用“脏矩形局部增量刷新技术”，成功用算法弥补了物理总线通信延迟的工程瓶颈，使得 Flappy Bird 游戏能以 30+ FPS 高频流畅运行，这加深了我对微机接口控制中时间开销与总线吞吐量限制的理解。
3. **提升了片上边缘智能与异常容错系统的设计能力**。通过引入指数滑动平均（EMA）数字滤波器和自学习环境温差判定算法，使得环境监测终端具备了自适应安装环境的能力。传感器热插拔自愈逻辑的实现，使得系统具备了工业级的容错能力。

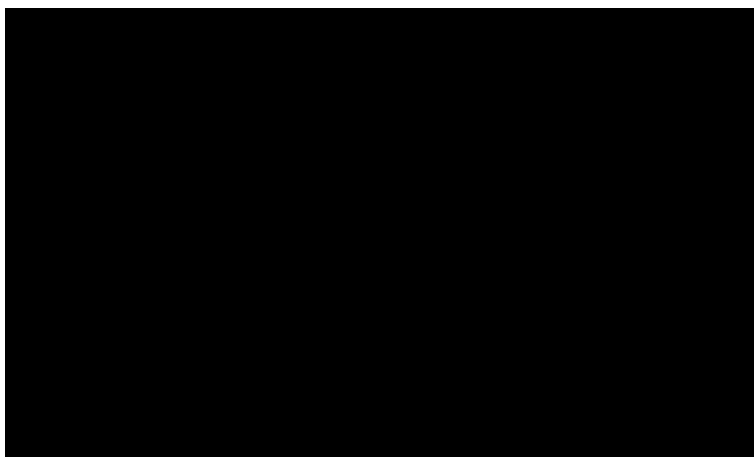


图 2: 环境温度自适应监测与娱乐交互系统实物测试现场（示意）

本报告记录的全部方案设计、底层代码实现与系统优化均由本人独立完成，相关实物运行现象完全符合设计预期，顺利通过实验验收。